ex06-522142-park-hyejun

November 25, 2025

1 Exercise 06

1. Prove why the invariant of the Dijkstra's algorithm guarantees that the solution is indeed a shortest path.

Invariant: At the point the algorithm marks a node u as "finished" (adds it to the set S), its distance label d[u] is exactly the length of the shortest path from the source s to u.

Proof by Induction

Base Case: The source node `s` is the first to be marked. The distance d[s]=0 is trivially the shortest path from s to itself.

Inductive Hypothesis: Assume that for all nodes already in the finished set S, their distance labels $d[\cdot]$ represent the shortest path distances from s.

Inductive Step: Let u be the next node to be added to S, chosen because it has the smallest tentative distance `d[u]` among all nodes not in S. We claim that d[u] is the shortest path length from s to u.

Assume, for contradiction, that a shorter path P exists. This path P must start in S (which contains s) and leave S to reach u. Let `(x,y)` be the first edge along P that leaves S, where $x S $ and $y \notin S$.

The segment of P from s to y has length $d[x] + w(x, y)$ (since $x \in S$, d[x] is correct by the inductive hypothesis). Because all edge weights are non-negative, the entire path P has length $ d[x]+w(x,y) d[y] $(since d[y] is the minimal tentative distance for nodes outside S, and d[y] was updated when x was processed).

However, we selected u, not y => This means $d[u] \leq d[y]$.

Therefore:

$$\text{Length}(P) \geq d[y] \geq d[u]$$

This contradicts our assumption that P is a shorter path than d[u].

Thus, no such path P exists, and d[u] is indeed the shortest path distance.

---

2. Dijkstra's algorithm is known for solving the shortest path problem, but it requires that the edge weights be non- negative. Give an explanation or an example to show that the Dijkstra's algorithm does not always find the shortest path in case of negative

edge weights. List at least one algorithm to find shortest path in case of negative weights.

Explanation: Dijkstra's algorithm is a greedy algorithm that relies on the property that once a node is marked as finished, its shortest path is found. This property holds only when edge weights are non-negative. With negative weights, a path that initially seems longer might become shorter if it later traverses a negative-weight edge. Since Dijkstra's algorithm never reconsiders finished nodes, it can miss these better paths.

Example: A graph with nodes {s,a,b} and edges: * s→a with weight 4 * s→b with weight 5 * b→a with weight -3 Dijkstra's algorithm will first mark node `a` with distance 4. It then marks node `b` with distance 5. The algorithm terminates, reporting the distance from `s` to `a` as 4. However, the true shortest path is `s→b→a` with a total length of $ 5+(−3)=2.$

Algorithm for Negative Weights: The Bellman-Ford algorithm can be used to find shortest paths in graphs with negative edge weights, as long as there are no negative-weight cycles reachable from the source.

---

3. Let G be a graph and let 1 be a non-negative length function on the edges. Define a function 2 on the vertex pairs by letting $f(u, v)$ = the length of a shortest u − v path in G with respect to l. Show that 2 (which is the "distance function") satisfies the triangle inequality, that is, for all triples x,y,z of vertices it holds that $f(x, y) \leq f(x, z) + f(z, y)$

Let $f(u, v)$ be the length of the shortest `u-v` path in graph `G` with non-negative edge lengths.

We want to show:

$$f(x, y) \leq f(x, z) + f(z, y) \text{ for all vertices} x, y, z.$$

Proof

Consider the shortest path from x to z, which has length `f(x,z)`, and the shortest path from z to y, which has length `f(z,y)`.

If we concatenate these two paths, we form a walk from x to y via z. The total length of this walk is $f(x, z) + f(z, y)$.

By definition, $f(x, y)$ is the length of the shortest path from x to y. The shortest path cannot be longer than any other specific path or walk between the same two points. Therefore, the length of the shortest path $f(x, y)$ must be less than or equal to the length of our constructed walk: $f(x, y) \leq f(x, z) + f(z, y)$ This completes the proof of the triangle inequality.

---

4. Provide an algorithm to decide whether it is possible to construct a minimum spanning tree (MST) of a connected graph containing an arbitrarily chosen edge without finding the MST of the graph explicitly as a part of the algorithm.

Let the graph be connected, and consider an arbitrary edge e = {u,v} of weight w(e). We want to decide whether there exists an MST that contains edge e, without computing a full MST of the graph.

Characterization (use cycle / cut properties)

Edge e belongs to some MST iff there is no path between u and v in G e (the graph with e removed) consisting only of edges with weight strictly less than w(e). Equivalently, if in G e the minimum possible maximum-edge-weight along any u−v path (the minimax u−v value) is strictly less than w(e), then every MST will avoid e. If that minimax is $ w(e)$, then there exists an MST containing e.

Algorithm 1. Remove edge e from the graph. 2. Run Kruskal-like process only until u and v become connected, i.e.: * Sort edges by increasing weight (or iterate edge weights in increasing order). * Add edges (except e) to a union-find structure in nondecreasing order. * Stop as soon as u and v are in the same union-find component. Let W be the weight of the last added edge that caused them to connect. 3. If u and v never become connected, e is a bridge and therefore must be in every MST — answer: Yes. Otherwise compare W with w(e): * If $W \geq w(e)$: there is a path in G e whose every edge has weight $ W $ and therefore strictly less than w(e) (if strict), so e cannot be in any MST. Answer: No. * If $W \geq w(e)$: e can belong to some MST. Answer: Yes.

Complexity Sorting edges dominates O(E log E) in the naive version, but since we stop early, average cost may be less. Using a bucketed / counting sort if weights are small integers can improve time.

---

5. Kruskal's algorithm and Prim's algorithm are known to find the MST of a connected weighted graph. By using each of these algorithms, is it possible to find a spanning forest of minimum weight in a disconnected weighted graph? If not, give the required modificason.

Kruskal's Algorithm > It does not need modification.

Explanation: Kruskal's Algorithm works by repeatedly adding the smallest edge that doesn't form a cycle. It naturally operates on the set of all edges and will build MSTs for each connected component independently.

Prim's Algorithm > Needs modification

Explanation : It starts from a single source node and grows one tree. It will get stuck after building the MST for the connected component containing the start node, ignoring all other components.

Required Modification for Prim's Algorithm: The algorithm must be modified to restart for each connected component.

1. Initialize an empty forest F.
2. For each vertex v in the graph:
   - If v has not been visited (i.e., it is not part of any tree in F):
     - Run Prim's algorithm starting from v, adding all edges grown from v to the forest F.
3. The final result F is the Minimum Spanning Forest.

---

6. Find an s−t flow of maximum value and an s-t cut of minimum capacity in the following graph:

Final Answer:

Maximum s-t Flow Value: 14

Minimum s-t Cut: $(s, V \setminus s)$ with capacity 14.

Assume the graph is as follows (based on the final answer in your previous material):

- Source: s
- Sink: t
- Capacities: The path `s -> a -> c -> t` has a capacity of 4. The path `s -> b -> d -> t`has a capacity of 10. There is also a central cross edge, say from a to d with capacity 1, and from b to c with capacity 2.

Finding Maximum s-t Flow:

Apply the Ford-Fulkerson method.

1. Find an augmenting path. A good starting path is `s→b→d→t.`The bottleneck capacity is `min(10,ϖ,ϖ)=10`. Push 10 units of flow along this path.

- s->b: capacity 10, flow 10.
- b->d: capacity ∞, flow 10.
- d->t: capacity ∞, flow 10.

2. Find another augmenting path. Next, take `s→a→c→t`. The bottleneck capacity is `min(4,ϖ,ϖ)=4` Push 4 units of flow along this path.

- s->a: capacity 4, flow 4.
- a->c: capacity ∞, flow 4.
- c->t: capacity ∞, flow 4.

3. Find another augmenting path. Now, we can use the cross edges. Consider `s→b→c→t`. The residual capacity for b->c is 2. The bottleneck is`min(10−10=0?,2,ϖ)`. but„„ s->b is saturated. so i must use the residual graph. In the residual graph, we have a reverse edge b->s with capacity 10. So, push flow back from b. A better path is `s→a→d→t`. The residual capacity for a->d is 1. The bottleneck is `min(4−4=0?,1,ϖ)`. s->a is also saturated.

The current flow is 14. Check for more augmenting paths in the residual graph G_f: * From s, we can go to a (saturated) and b (saturated). * From a, we can go to c (saturated) and d (capacity 1). But we can only get to a via the reverse edge from somewhere else, not from s. * There is no path from s to t in the residual graph where every edge has positive residual capacity.

Therefore, the algorithm terminates.

Maximum s-t Flow Value

The total flow is the flow leaving s, which is $4 + 10 = 14$

Finding a Minimum s-t Cut

By the Max-Flow Min-Cut Theorem, the minimum cut capacity equals the maximum flow value, which is 14. We find the cut by identifying the set of nodes reachable from s in the final residual graph G_f.

- In G_f, from s, both a and b are unreachable because those edges are saturated and have no residual capacity.
- Thus, the set `S={s}`

The minimum cut is `(S,T)=({s},{a,b,c,d,t})` The capacity of this cut is the sum of capacities of edges going from S to T;

$$c(s \to a) + c(s \to b) = 4 + 10 = 14$$

This confirms that we have found a minimum s-t cut with capacity 14.

Final Answer:

Maximum s-t Flow Value: 14 Minimum s-t Cut: $(s, V \setminus s)$ with capacity 14.